

CSIT751 (Core Java)

Programming Exercises for Practice (Practical Home Assignment)

Module III – Exception Handling, Interface and Thread in Java

- Q1. ATM Withdrawal System – Handling Insufficient Balance
- Q2. Student Age Validation – User Defined Exception
- Q3. Array Division – Handling Arithmetic and Array Index Exception (Multiple Catch)
- Q4. Student Marks Entry – Input Validation
- Q5. University Login System – Null Pointer Handling
- Q6. Online Shopping – Minimum Purchase Amount Exception
- Q7. Railway Ticket Booking – Synchronization of Threads
- Q8. University Printing System – Multiple Threads
- Q9. IoT Sensor Monitoring System
- Q10. Bank Transaction System – Thread Priorities

Module IV – Collections and GUI Programming

- Q1. Library Book Management – Using ArrayList
- Q2. Student Attendance System – Using HashSet
- Q3. Bank Account Directory – Using HashMap
- Q4. Product Inventory Management – Using HashMap and Iterator
- Q5. Online Course Enrollment – Using LinkedHashSet
- Q6. Student Registration Form — GUI with AWT Components
- Q7. Calculator Form — Event-Driven Programming

Each question shows the problem statement, the Java source code, and the actual output captured from running the program in a terminal.

Module III – Exception Handling, Interface and Thread in Java

Q1. ATM Withdrawal System – Handling Insufficient Balance

Scenario: In a banking system, when a user tries to withdraw more money than their account balance, the system should throw an exception and display an appropriate message.

PROGRAM STRUCTURE

- Class: **ATM** with field **balance** and method **withdraw(double amount)** — throws an exception if amount > balance
- Use **throw** and **try-catch**
- Use a **finally** block to display "Transaction Completed"

CODE • ATM.java

```
import java.util.Scanner;

public class ATM {
    private double balance;

    ATM(double balance) {
        this.balance = balance;
    }

    void withdraw(double amount) throws Exception {
        if (amount > balance) {
            throw new Exception("Insufficient Balance");
        }
        balance -= amount;
        System.out.println("Withdrawn: " + amount + ", Remaining balance: " + balance);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter balance: ");
        double balance = sc.nextDouble();
        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        ATM atm = new ATM(balance);
        try {
            atm.withdraw(amount);
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Transaction Completed");
        }
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac ATM.java
student@java-lab:~/module3$ java ATM
Enter balance: 5000
Enter amount to withdraw: 6000
Exception: Insufficient Balance
Transaction Completed

student@java-lab:~/module3$ java ATM
Enter balance: 5000
Enter amount to withdraw: 1500
Withdrawn: 1500.0, Remaining balance: 3500.0
Transaction Completed
student@java-lab:~/module3$ █
```

Q2. Student Age Validation - User Defined Exception

Scenario: While registering students, the age must be above 17. If not, throw a user-defined exception.

PROGRAM STRUCTURE

- Class: **Student**
- User defined exception: **InvalidAgeException**
- Method: **registerStudent(int age)** — throws InvalidAgeException for an invalid age
- Handle using try-catch

CODE • Student.java

```
import java.util.Scanner;

// User-defined (checked) exception
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

public class Student {
    private String name;

    Student(String name) {
        this.name = name;
    }

    // Age must be above 17, so 17 and below are rejected
    void registerStudent(int age) throws InvalidAgeException {
        if (age <= 17) {
            throw new InvalidAgeException("Age must be above 17 for registration");
        }
        System.out.println(name + " registered successfully");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.print("Enter age: ");
        int age = sc.nextInt();

        Student s = new Student(name);
        try {
            s.registerStudent(age);
        } catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac Student.java
student@java-lab:~/module3$ java Student
Enter name: Rohan
Enter age: 15
Exception: Age must be above 17 for registration

student@java-lab:~/module3$ java Student
Enter name: Aditi
Enter age: 20
Aditi registered successfully
student@java-lab:~/module3$
```

Q3. Array Division - Handling Arithmetic and Array Index Exception (Multiple Catch)

Scenario: A program takes two numbers and an array index. It divides array[index] by the number. Handle both ArithmeticException (divisor = 0) and ArrayIndexOutOfBoundsException (invalid index).

PROGRAM STRUCTURE

- Class: **ArrayDivision**
- Use **multiple catch** blocks

CODE · ArrayDivision.java

```
import java.util.Arrays;
import java.util.Scanner;

public class ArrayDivision {
    public static void main(String[] args) {
        int[] array = {10, 20, 30, 40};
        System.out.println("Array: " + Arrays.toString(array));

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter index: ");
        int index = sc.nextInt();
        System.out.print("Enter divisor: ");
        int divisor = sc.nextInt();

        try {
            int result = array[index] / divisor;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception: Division by zero not allowed");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception: Array index out of bounds");
        }
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac ArrayDivision.java
student@java-lab:~/module3$ java ArrayDivision
Array: [10, 20, 30, 40]
Enter index: 5
Enter divisor: 2
Exception: Array index out of bounds

student@java-lab:~/module3$ java ArrayDivision
Array: [10, 20, 30, 40]
Enter index: 2
Enter divisor: 0
Exception: Division by zero not allowed

student@java-lab:~/module3$ java ArrayDivision
Array: [10, 20, 30, 40]
Enter index: 3
Enter divisor: 4
Result: 10
student@java-lab:~/module3$
```

Q4. Student Marks Entry - Input Validation

Scenario: If a user enters a negative mark or a mark greater than 100, throw an exception. Also ensure that the program always displays a thank you message using finally.

PROGRAM STRUCTURE

- Class: **StudentMarks**
- Method: **enterMarks(int marks)** — throws exception for invalid range
- Use **try-catch-finally**

CODE · StudentMarks.java

```
import java.util.Scanner;

public class StudentMarks {
    void enterMarks(int marks) throws Exception {
        if (marks < 0 || marks > 100) {
            throw new Exception("Marks must be between 0 and 100");
        }
        System.out.println("Marks accepted: " + marks);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        StudentMarks sm = new StudentMarks();
        try {
            sm.enterMarks(marks);
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Thank you for using the system");
        }
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac StudentMarks.java
student@java-lab:~/module3$ java StudentMarks
Enter marks: 105
Exception: Marks must be between 0 and 100
Thank you for using the system

student@java-lab:~/module3$ java StudentMarks
Enter marks: 78
Marks accepted: 78
Thank you for using the system
student@java-lab:~/module3$
```

Q5. University Login System - Null Pointer Handling

Scenario: In a university login system, if a user tries to log in with a null username, a `NullPointerException` occurs. Handle this gracefully.

PROGRAM STRUCTURE

- Class: **UniversityLogin**
- Method: **login(String username)**
- Throw or let `NullPointerException` occur and handle it using try-catch

CODE • UniversityLogin.java

```
import java.util.Scanner;

public class UniversityLogin {
    void login(String username) {
        // Calling a method on a null reference throws NullPointerException
        System.out.println("Welcome, " + username.toUpperCase());
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter username: ");
        String input = sc.nextLine().trim();
        String username = input.equals("null") ? null : input;

        UniversityLogin ul = new UniversityLogin();
        try {
            ul.login(username);
        } catch (NullPointerException e) {
            System.out.println("Exception: Username cannot be null");
        }
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac UniversityLogin.java
student@java-lab:~/module3$ java UniversityLogin
Enter username: null
Exception: Username cannot be null

student@java-lab:~/module3$ java UniversityLogin
Enter username: rohan
Welcome, ROHAN
student@java-lab:~/module3$
```

Q6. Online Shopping - Minimum Purchase Amount Exception

Scenario: An online shopping system requires a minimum cart value of ₹500 for placing an order. If below that, throw a user-defined exception.

PROGRAM STRUCTURE

- Class: **OnlineShopping**
- User defined exception: **MinimumAmountException**
- Method: **placeOrder(int amount)** — throws exception if amount < 500

CODE · OnlineShopping.java

```
import java.util.Scanner;

// User-defined (checked) exception
class MinimumAmountException extends Exception {
    MinimumAmountException(String message) {
        super(message);
    }
}

public class OnlineShopping {
    void placeOrder(int amount) throws MinimumAmountException {
        if (amount < 500) {
            throw new MinimumAmountException("Minimum cart value must be ₹500");
        }
        System.out.println("Order placed successfully for ₹" + amount);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter cart amount: ");
        int amount = sc.nextInt();

        OnlineShopping shop = new OnlineShopping();
        try {
            shop.placeOrder(amount);
        } catch (MinimumAmountException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac OnlineShopping.java
student@java-lab:~/module3$ java OnlineShopping
Enter cart amount: 300
Exception: Minimum cart value must be ₹500

student@java-lab:~/module3$ java OnlineShopping
Enter cart amount: 750
Order placed successfully for ₹750
student@java-lab:~/module3$
```

Q7. Railway Ticket Booking - Synchronization of Threads

Scenario: In a railway booking system, multiple users are trying to book tickets simultaneously. Only a limited number of seats are available. Synchronization must ensure that no two users book the same seat.

PROGRAM STRUCTURE

- Class: **TicketBooking** (shared resource) with field **availableSeats** and **synchronized** method **bookSeat(int seats)**
- Thread classes: **User1**, **User2** extending Thread
- Use synchronization to prevent race conditions

CODE • TicketBooking.java

```
import java.util.Scanner;

// Shared resource
public class TicketBooking {
    private int availableSeats;

    TicketBooking(int availableSeats) {
        this.availableSeats = availableSeats;
    }

    // synchronized: only one thread can book at a time (no race condition)
    synchronized void bookSeat(String user, int seats) {
        if (seats <= availableSeats) {
            availableSeats -= seats;
            System.out.println(user + " booked " + seats + " seat(s) successfully");
        } else {
            System.out.println(user + " booking failed. Not enough seats");
        }
    }

    int getAvailableSeats() {
        return availableSeats;
    }

    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        System.out.print("Available Seats: ");
        int seats = sc.nextInt();
        System.out.print("User1 wants to book: ");
        int s1 = sc.nextInt();
        System.out.print("User2 wants to book: ");
        int s2 = sc.nextInt();

        TicketBooking tb = new TicketBooking(seats);
        User1 u1 = new User1(tb, s1);
        User2 u2 = new User2(tb, s2);

        u1.start();
        Thread.sleep(200); // small gap so the output order is predictable
        u2.start();

        u1.join();
        u2.join();
        System.out.println("Seats left: " + tb.getAvailableSeats());
    }
}

class User1 extends Thread {
    private TicketBooking tb;
    private int seats;

    User1(TicketBooking tb, int seats) {
        this.tb = tb;
        this.seats = seats;
    }

    public void run() {
        tb.bookSeat("User1", seats);
    }
}
```

```
    }  
}  
  
class User2 extends Thread {  
    private TicketBooking tb;  
    private int seats;  
  
    User2(TicketBooking tb, int seats) {  
        this.tb = tb;  
        this.seats = seats;  
    }  
  
    public void run() {  
        tb.bookSeat("User2", seats);  
    }  
}
```

OUTPUT

```
student@java-lab:~/module3$ javac TicketBooking.java  
student@java-lab:~/module3$ java TicketBooking  
Available Seats: 2  
User1 wants to book: 1  
User2 wants to book: 2  
User1 booked 1 seat(s) successfully  
User2 booking failed. Not enough seats  
Seats left: 1  
  
student@java-lab:~/module3$ java TicketBooking  
Available Seats: 5  
User1 wants to book: 2  
User2 wants to book: 3  
User1 booked 2 seat(s) successfully  
User2 booked 3 seat(s) successfully  
Seats left: 0  
student@java-lab:~/module3$
```

Q8. University Printing System - Multiple Threads

Scenario: A university has multiple printers. Students can send print jobs simultaneously. Each print job runs on a separate thread.

PROGRAM STRUCTURE

- Class: **PrinterJob** implements Runnable
- Create multiple threads for different students
- Simulate printing using **Thread.sleep()**

CODE · PrinterJob.java

```
import java.util.Scanner;

public class PrinterJob implements Runnable {
    private final int jobNo;
    private final String student;

    PrinterJob(int jobNo, String student) {
        this.jobNo = jobNo;
        this.student = student;
    }

    public void run() {
        System.out.println("Printing job " + jobNo + " by " + student);
        try {
            Thread.sleep(1000); // simulate printing time
        } catch (InterruptedException e) {
            System.out.println("Job " + jobNo + " interrupted");
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        System.out.print("Number of print jobs: ");
        int n = sc.nextInt();

        Thread[] jobs = new Thread[n];
        for (int i = 0; i < n; i++) {
            String student = "Student " + (char) ('A' + i);
            jobs[i] = new Thread(new PrinterJob(i + 1, student));
            jobs[i].start();
            Thread.sleep(300);
        }
        for (Thread t : jobs) {
            t.join();
        }
        System.out.println("All print jobs completed");
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac PrinterJob.java
student@java-lab:~/module3$ java PrinterJob
Number of print jobs: 3
Printing job 1 by Student A
Printing job 2 by Student B
Printing job 3 by Student C
All print jobs completed
student@java-lab:~/module3$
```

Q9. IoT Sensor Monitoring System

Scenario: An IoT sensor generates a random number every second. If even, the square is calculated by one thread. If odd, the cube is calculated by another thread. Menu: 1. Start Simulation, 2. Exit.

PROGRAM STRUCTURE

- **RandomNumberGenerator** (Thread) → generates numbers
- **SquareCalculator** (Thread) → processes even numbers
- **CubeCalculator** (Thread) → processes odd numbers
- Uses Thread.sleep(), multithreading and conditional checks

CODE • IoTSensorMonitor.java

```
import java.util.Random;
import java.util.Scanner;

class SquareCalculator extends Thread {
    private int n;

    SquareCalculator(int n) {
        this.n = n;
    }

    public void run() {
        System.out.println("Square: " + (n * n));
    }
}

class CubeCalculator extends Thread {
    private int n;

    CubeCalculator(int n) {
        this.n = n;
    }

    public void run() {
        System.out.println("Cube: " + (n * n * n));
    }
}

class RandomNumberGenerator extends Thread {
    private int count;

    RandomNumberGenerator(int count) {
        this.count = count;
    }

    public void run() {
        Random random = new Random();
        try {
            for (int i = 0; i < count; i++) {
                int n = random.nextInt(20) + 1;
                System.out.println("Generated: " + n);

                Thread worker = (n % 2 == 0) ? new SquareCalculator(n) : new CubeCalculator(n);
                worker.start();
                worker.join();

                Thread.sleep(1000); // sensor reading every second
            }
        } catch (InterruptedException e) {
            System.out.println("Sensor interrupted");
        }
    }
}

public class IoTSensorMonitor {
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("1. Start Simulation");
        }
    }
}
```

```
System.out.println("2. Exit");
System.out.print("Enter choice: ");
int choice = sc.nextInt();

if (choice == 1) {
    RandomNumberGenerator gen = new RandomNumberGenerator(6);
    gen.start();
    gen.join();
    System.out.println("Simulation finished");
} else if (choice == 2) {
    System.out.println("Exiting...");
    break;
} else {
    System.out.println("Invalid choice");
}
}
}
```

OUTPUT

```
student@java-lab:~/module3$ javac IoTSensorMonitor.java
student@java-lab:~/module3$ java IoTSensorMonitor
1. Start Simulation
2. Exit
Enter choice: 1
Generated: 8
Square: 64
Generated: 12
Square: 144
Generated: 19
Cube: 6859
Generated: 7
Cube: 343
Generated: 14
Square: 196
Generated: 2
Square: 4
Simulation finished
1. Start Simulation
2. Exit
Enter choice: 2
Exiting...
student@java-lab:~/module3$
```

Q10. Bank Transaction System - Thread Priorities

Scenario: In a banking application, high-value transactions must be processed with higher priority than low-value transactions.

PROGRAM STRUCTURE

- Class: **BankTransaction** extends Thread
- Create threads for different transactions and set priorities: **MAX_PRIORITY** for high-value, **MIN_PRIORITY** for low-value

CODE · BankTransaction.java

```
import java.util.Scanner;
import java.util.concurrent.atomic.AtomicInteger;

public class BankTransaction extends Thread {
    private static final double HIGH_VALUE = 10000;
    private static final AtomicInteger order = new AtomicInteger(0);

    private final double amount;

    BankTransaction(String name, double amount) {
        super(name);
        this.amount = amount;
        // High-value gets MAX_PRIORITY, low-value gets MIN_PRIORITY
        setPriority(amount >= HIGH_VALUE ? Thread.MAX_PRIORITY : Thread.MIN_PRIORITY);
    }

    public void run() {
        String type = amount >= HIGH_VALUE ? "High-value" : "Low-value";
        String when = (order.getAndIncrement() == 0) ? "first" : "later";
        System.out.println(type + " transaction processed " + when);
    }

    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        System.out.print("Transaction1: ₹");
        double a1 = sc.nextDouble();
        System.out.print("Transaction2: ₹");
        double a2 = sc.nextDouble();

        BankTransaction t1 = new BankTransaction("Transaction1", a1);
        BankTransaction t2 = new BankTransaction("Transaction2", a2);
        System.out.println("Priorities -> Transaction1: " + t1.getPriority()
            + ", Transaction2: " + t2.getPriority());

        // Priority is only a hint to the OS scheduler, so the higher priority
        // thread is started first to make the order predictable.
        BankTransaction firstT = t1.getPriority() >= t2.getPriority() ? t1 : t2;
        BankTransaction secondT = (firstT == t1) ? t2 : t1;

        firstT.start();
        Thread.sleep(200);
        secondT.start();

        firstT.join();
        secondT.join();
    }
}
```

OUTPUT

```
student@java-lab:~/module3$ javac BankTransaction.java
student@java-lab:~/module3$ java BankTransaction
Transaction1: ₹5000
Transaction2: ₹50000
Priorities -> Transaction1: 1, Transaction2: 10
High-value transaction processed first
Low-value transaction processed later
student@java-lab:~/module3$
```

Module IV – Collections and GUI Programming

Q1. Library Book Management - Using ArrayList

Scenario: A library maintains a list of book titles. The system should allow adding new books, removing books, and displaying the complete book list.

PROGRAM STRUCTURE

- Class: **Library** using **ArrayList<String>** to store book titles
- Methods: **addBook()**, **removeBook()**, **displayBooks()**
- Menu-driven program

Concepts: ArrayList operations (add, remove, iterate)

CODE • Library.java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Library {
    private ArrayList<String> books = new ArrayList<>();

    void addBook(String title) {
        books.add(title);
        System.out.println("Book added successfully.");
    }

    void removeBook(String title) {
        if (books.remove(title)) {
            System.out.println("Book removed successfully.");
        } else {
            System.out.println("Book not found.");
        }
    }

    void displayBooks() {
        if (books.isEmpty()) {
            System.out.println("No books in the library.");
            return;
        }
        System.out.println("Current Books:");
        for (String b : books) {
            System.out.println(b);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library lib = new Library();

        while (true) {
            System.out.println("1. Add Book");
            System.out.println("2. Remove Book");
            System.out.println("3. Display All Books");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
                case 1:
                    System.out.print("Enter book title: ");
                    lib.addBook(sc.nextLine());
                    break;
                case 2:
                    System.out.print("Enter book title to remove: ");
                    lib.removeBook(sc.nextLine());
                    break;
                case 3:
                    lib.displayBooks();
                    break;
                case 4:
                    return;
            }
        }
    }
}
```

```
        break;
    case 4:
        System.out.println("Goodbye!");
        return;
    default:
        System.out.println("Invalid choice.");
    }
}
}
```

OUTPUT

```
student@java-lab:~/module4$ javac Library.java
student@java-lab:~/module4$ java Library
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 1
Enter book title: Data Structures
Book added successfully.
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 1
Enter book title: Java Programming
Book added successfully.
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 3
Current Books:
Data Structures
Java Programming
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 2
Enter book title to remove: Data Structures
Book removed successfully.
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 3
Current Books:
Java Programming
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 4
Goodbye!
student@java-lab:~/module4$
```

Q2. Student Attendance System - Using HashSet

Scenario: In a classroom, attendance must not contain duplicates. Using a Set ensures each student is marked only once.

PROGRAM STRUCTURE

- Class: **Attendance** using **HashSet<String>** to store student names
- Methods: **markAttendance()**, **displayAttendance()**
- Prevent duplicate entries

Concepts: Set, uniqueness, no duplicates

CODE • Attendance.java

```
import java.util.HashSet;
import java.util.Scanner;

public class Attendance {
    private HashSet<String> present = new HashSet<>();

    void markAttendance(String name) {
        // add() returns false if the element already exists
        if (present.add(name)) {
            System.out.println("Attendance marked for " + name);
        } else {
            System.out.println(name + " is already marked present");
        }
    }

    void displayAttendance() {
        System.out.println("Present students: " + present);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Attendance att = new Attendance();

        while (true) {
            System.out.print("Enter name to mark attendance (type 'exit' to finish): ");
            String name = sc.nextLine().trim();
            if (name.equalsIgnoreCase("exit")) {
                break;
            }
            att.markAttendance(name);
        }
        att.displayAttendance();
    }
}
```

OUTPUT

```
student@java-lab:~/module4$ javac Attendance.java
student@java-lab:~/module4$ java Attendance
Enter name to mark attendance (type 'exit' to finish): Rohan
Attendance marked for Rohan
Enter name to mark attendance (type 'exit' to finish): Rohan
Rohan is already marked present
Enter name to mark attendance (type 'exit' to finish): Aditi
Attendance marked for Aditi
Enter name to mark attendance (type 'exit' to finish): exit
Present students: [Aditi, Rohan]
student@java-lab:~/module4$
```

Q3. Bank Account Directory - Using HashMap

Scenario: A bank maintains a directory mapping account numbers to customer names. It should allow adding accounts, retrieving customer name by account number, and displaying all entries.

PROGRAM STRUCTURE

- Class: **BankDirectory** using **HashMap<Integer, String>**
- Methods: **addAccount(), getCustomer(), displayAll()**

Concepts: Map, key-value pair, retrieval

CODE · BankDirectory.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class BankDirectory {
    private HashMap<Integer, String> accounts = new HashMap<>();

    void addAccount(int accNo, String name) {
        accounts.put(accNo, name);
        System.out.println("Account added successfully.");
    }

    void getCustomer(int accNo) {
        String name = accounts.get(accNo);
        if (name != null) {
            System.out.println("Customer Name: " + name);
        } else {
            System.out.println("Account not found.");
        }
    }

    void displayAll() {
        if (accounts.isEmpty()) {
            System.out.println("No accounts available.");
            return;
        }
        for (Map.Entry<Integer, String> e : accounts.entrySet()) {
            System.out.println("Account No: " + e.getKey() + " → " + e.getValue());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankDirectory bank = new BankDirectory();

        while (true) {
            System.out.println("1. Add Account");
            System.out.println("2. Get Customer Name");
            System.out.println("3. Display All Accounts");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
                case 1:
                    System.out.print("Enter Account No: ");
                    int no = Integer.parseInt(sc.nextLine().trim());
                    System.out.print("Enter Customer Name: ");
                    bank.addAccount(no, sc.nextLine());
                    break;
                case 2:
                    System.out.print("Enter Account No: ");
                    bank.getCustomer(Integer.parseInt(sc.nextLine().trim()));
                    break;
                case 3:
                    bank.displayAll();
                    break;
                case 4:
                    System.out.println("Goodbye!");
            }
        }
    }
}
```

```
        return;  
    default:  
        System.out.println("Invalid choice.");  
    }  
}  
}  
}
```

OUTPUT

```
student@java-lab:~/module4$ javac BankDirectory.java  
student@java-lab:~/module4$ java BankDirectory  
1. Add Account  
2. Get Customer Name  
3. Display All Accounts  
4. Exit  
Enter choice: 1  
Enter Account No: 101  
Enter Customer Name: Ankit  
Account added successfully.  
1. Add Account  
2. Get Customer Name  
3. Display All Accounts  
4. Exit  
Enter choice: 1  
Enter Account No: 102  
Enter Customer Name: Priya  
Account added successfully.  
1. Add Account  
2. Get Customer Name  
3. Display All Accounts  
4. Exit  
Enter choice: 2  
Enter Account No: 101  
Customer Name: Ankit  
1. Add Account  
2. Get Customer Name  
3. Display All Accounts  
4. Exit  
Enter choice: 2  
Enter Account No: 999  
Account not found.  
1. Add Account  
2. Get Customer Name  
3. Display All Accounts  
4. Exit  
Enter choice: 3  
Account No: 101 → Ankit  
Account No: 102 → Priya  
1. Add Account  
2. Get Customer Name  
3. Display All Accounts  
4. Exit  
Enter choice: 4  
Goodbye!  
student@java-lab:~/module4$
```

Q4. Product Inventory Management - Using HashMap and Iterator

Scenario: A shop maintains an inventory where each product ID maps to its stock count. The program should add products, update stock, and display inventory using an iterator.

PROGRAM STRUCTURE

- Class: **Inventory** using **HashMap<Integer, Integer>** for product ID → stock
- Methods: **addProduct(), updateStock(), displayInventory()**

Concepts: HashMap, Iterator traversal

CODE · Inventory.java

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Scanner;

public class Inventory {
    private HashMap<Integer, Integer> stock = new HashMap<>();

    void addProduct(int id, int qty) {
        stock.put(id, qty);
        System.out.println("Product " + id + " added with stock " + qty);
    }

    void updateStock(int id, int newQty) {
        if (stock.containsKey(id)) {
            int old = stock.put(id, newQty);
            System.out.println("Stock updated for product " + id + ": " + old + " -> " + newQty);
        } else {
            System.out.println("Product " + id + " not found");
        }
    }

    void displayInventory() {
        Iterator<Map.Entry<Integer, Integer>> it = stock.entrySet().iterator();
        if (!it.hasNext()) {
            System.out.println("Inventory is empty.");
            return;
        }
        while (it.hasNext()) {
            Map.Entry<Integer, Integer> e = it.next();
            System.out.println("Product ID: " + e.getKey() + " | Stock: " + e.getValue());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Inventory inv = new Inventory();

        while (true) {
            System.out.println("1. Add Product");
            System.out.println("2. Update Stock");
            System.out.println("3. Display Inventory");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
                case 1:
                    System.out.print("Enter Product ID: ");
                    int id = Integer.parseInt(sc.nextLine().trim());
                    System.out.print("Enter Stock: ");
                    inv.addProduct(id, Integer.parseInt(sc.nextLine().trim()));
                    break;
                case 2:
                    System.out.print("Enter Product ID: ");
                    int pid = Integer.parseInt(sc.nextLine().trim());
                    System.out.print("Enter New Stock: ");
                    inv.updateStock(pid, Integer.parseInt(sc.nextLine().trim()));
                    break;
            }
        }
    }
}
```

```

        case 3:
            inv.displayInventory();
            break;
        case 4:
            System.out.println("Goodbye!");
            return;
        default:
            System.out.println("Invalid choice.");
    }
}
}
}
}
}

```

OUTPUT

```

student@java-lab:~/module4$ javac Inventory.java
student@java-lab:~/module4$ java Inventory
1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 1
Enter Product ID: 101
Enter Stock: 50
Product 101 added with stock 50
1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 1
Enter Product ID: 102
Enter Stock: 30
Product 102 added with stock 30
1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 2
Enter Product ID: 101
Enter New Stock: 75
Stock updated for product 101: 50 -> 75
1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 3
Product ID: 101 | Stock: 75
Product ID: 102 | Stock: 30
1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 4
Goodbye!
student@java-lab:~/module4$ 

```

Q5. Online Course Enrollment - Using LinkedHashSet

Scenario: Students enroll in online courses. The enrollment list must be stored in the order of registration but without duplicates. LinkedHashSet is ideal here.

PROGRAM STRUCTURE

- Class: **CourseEnrollment** using **LinkedHashSet<String>**
- Methods: **enrollStudent()**, **displayEnrolledStudents()**

Concepts: LinkedHashSet (ordered + unique), hashing

CODE · CourseEnrollment.java

```
import java.util.LinkedHashSet;
import java.util.Scanner;

public class CourseEnrollment {
    // Keeps registration order and rejects duplicates
    private LinkedHashSet<String> enrolled = new LinkedHashSet<>();

    void enrollStudent(String name) {
        if (enrolled.add(name)) {
            System.out.println("Enrolled: " + name);
        } else {
            System.out.println(name + " is already enrolled");
        }
    }

    void displayEnrolledStudents() {
        System.out.println("Enrolled Students: " + enrolled);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CourseEnrollment course = new CourseEnrollment();

        while (true) {
            System.out.print("Enroll (type 'done' to finish): ");
            String name = sc.nextLine().trim();
            if (name.equalsIgnoreCase("done")) {
                break;
            }
            course.enrollStudent(name);
        }
        course.displayEnrolledStudents();
    }
}
```

OUTPUT

```
student@java-lab:~/module4$ javac CourseEnrollment.java
student@java-lab:~/module4$ java CourseEnrollment
Enroll (type 'done' to finish): Aditi
Enrolled: Aditi
Enroll (type 'done' to finish): Rohan
Enrolled: Rohan
Enroll (type 'done' to finish): Aditi
Aditi is already enrolled
Enroll (type 'done' to finish): done
Enrolled Students: [Aditi, Rohan]
student@java-lab:~/module4$
```

Q6. Student Registration Form — GUI with AWT Components

Scenario: Create a student registration form using AWT components. The form should accept basic student details and display them after submission.

PROGRAM STRUCTURE

- Use **Frame** as container
- Add labels and text fields for Name, Roll No, and Course (the form view also shows Father Name, Age, Gender, Hobbies and Address)
- Add a **Submit** button (Save Details); on click, display the entered data in a dialog box or on the console
- Use **FlowLayout**

CODE · RegistrationForm.java

```
import java.awt.*;
import java.awt.event.*;

public class RegistrationForm extends Frame implements ActionListener {
    static final Color BG = new Color(0x38, 0x3D, 0x4A);

    TextField tfName = new TextField(28);
    TextField tfRoll = new TextField(28);
    TextField tfFather = new TextField(28);
    TextField tfAge = new TextField(28);

    CheckboxGroup genderGroup = new CheckboxGroup();
    Checkbox male = new Checkbox("Male", genderGroup, true);
    Checkbox female = new Checkbox("Female", genderGroup, false);

    Choice course = new Choice();

    Checkbox drawing = new Checkbox("Drawing");
    Checkbox singing = new Checkbox("Singing");
    Checkbox music = new Checkbox("Music");
    Checkbox others = new Checkbox("Others");

    TextArea address = new TextArea(4, 28);

    Button save = new Button("Save Details");
    Button clear = new Button("Clear All");

    RegistrationForm() {
        super("Student Registration Form");
        setLayout(new FlowLayout(FlowLayout.LEFT, 20, 8));
        setBackground(BG);

        Label title = new Label("Registration Form");
        title.setFont(new Font("SansSerif", Font.BOLD, 18));
        title.setForeground(Color.YELLOW);
        add(title);

        course.add("Java");
        course.add("Python");
        course.add("C++");
        course.add("Web Development");

        Panel genderPanel = new Panel(new FlowLayout(FlowLayout.LEFT, 10, 0));
        genderPanel.add(male);
        genderPanel.add(female);

        Panel hobbyPanel = new Panel(new FlowLayout(FlowLayout.LEFT, 5, 0));
        hobbyPanel.add(drawing);
        hobbyPanel.add(singing);
        hobbyPanel.add(music);
        hobbyPanel.add(others);

        Panel buttons = new Panel(new FlowLayout(FlowLayout.LEFT, 10, 0));
        save.setBackground(Color.BLUE);
        save.setForeground(Color.WHITE);
        clear.setBackground(Color.RED);
    }
}
```

```

clear.setForeground(Color.WHITE);
buttons.add(save);
buttons.add(clear);

addRow("Name", tfName);
addRow("Roll No", tfRoll);
addRow("Father Name", tfFather);
addRow("Age", tfAge);
addRow("Gender", genderPanel);
addRow("Course", course);
addRow("Hobbies", hobbyPanel);
addRow("Address", address);
addRow("", buttons);

save.addActionListener(this);
clear.addActionListener(this);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

setSize(520, 640);
setLocationRelativeTo(null);
}

// One row = label + component, laid out with FlowLayout
private void addRow(String text, Component c) {
    Panel row = new Panel(new FlowLayout(FlowLayout.LEFT, 5, 2));
    row.setBackground(BG);
    Label l = new Label(text) {
        public Dimension getPreferredSize() {
            return new Dimension(100, 22);
        }
    };
    l.setForeground(Color.WHITE);
    row.add(l);
    row.add(c);
    c.setBackground(c instanceof Panel ? BG : Color.WHITE);
    if (c instanceof Panel) {
        for (Component child : ((Panel) c).getComponents()) {
            if (child instanceof Checkbox) {
                child.setForeground(Color.WHITE);
                child.setBackground(BG);
            }
        }
    }
    add(row);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == clear) {
        tfName.setText("");
        tfRoll.setText("");
        tfFather.setText("");
        tfAge.setText("");
        male.setState(true);
        course.select(0);
        drawing.setState(false);
        singing.setState(false);
        music.setState(false);
        others.setState(false);
        address.setText("");
        return;
    }
}

StringBuilder hobbies = new StringBuilder();
if (drawing.getState()) hobbies.append("Drawing ");
if (singing.getState()) hobbies.append("Singing ");
if (music.getState()) hobbies.append("Music ");
if (others.getState()) hobbies.append("Others ");

```

```

String details = "Name      : " + tfName.getText() + "\n"
                + "Roll No   : " + tfRoll.getText() + "\n"
                + "Father Name: " + tfFather.getText() + "\n"
                + "Age       : " + tfAge.getText() + "\n"
                + "Gender    : " + genderGroup.getSelectedCheckbox().getLabel() + "\n"
                + "Course    : " + course.getSelectedItem() + "\n"
                + "Hobbies   : " + hobbies.toString().trim() + "\n"
                + "Address   : " + address.getText().trim().replaceAll(",?\s*\r?\n", ", ");

System.out.println("Registration successful\n" + details);
showDialog(details);
}

private void showDialog(String details) {
    final Dialog d = new Dialog(this, "Registration", true);
    d.setLayout(new BorderLayout(8, 8));
    Label msg = new Label("Student registered successfully!", Label.CENTER);
    msg.setFont(new Font("SansSerif", Font.BOLD, 13));
    d.add(msg, BorderLayout.NORTH);

    TextArea ta = new TextArea(details, 8, 58, TextArea.SCROLLBARS_NONE);
    ta.setEditable(false);
    ta.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
    d.add(ta, BorderLayout.CENTER);

    Button ok = new Button("OK");
    ok.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent ev) {
            d.dispose();
        }
    });
    Panel south = new Panel();
    south.add(ok);
    d.add(south, BorderLayout.SOUTH);

    d.addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent ev) {
            d.dispose();
        }
    });
    d.setSize(520, 270);
    d.setLocationRelativeTo(this);
    d.setVisible(true);
}

public static void main(String[] args) {
    new RegistrationForm().setVisible(true);
}
}

```

OUTPUT

```
student@java-lab:~/module4$ javac RegistrationForm.java
student@java-lab:~/module4$ java RegistrationForm
# GUI window opens; after clicking "Save Details":
Registration successful
Name       : Ram
Roll No    : 101
Father Name: Kumar
Age        : 23
Gender     : Male
Course     : Java
Hobbies    : Drawing
Address    : 234 - 1d First Street, Anna Main Road, Namakkal
student@java-lab:~/module4$
```

Registration Form

Name

Roll No

Father Name

Age

Gender ☐ Male ☐ Female

Course

Hobbies ☐ Drawing ☐ Singing ☐ Music ☐ Others

Address

Form on start

Registration Form

Name

Roll No

Father Name

Age

Gender ☐ Male ☐ Female

Course

Hobbies ☒ Drawing ☐ Singing ☐ Music ☐ Others

Address

Form filled in

Registration Form

Name :

Roll No :

Father Name :

Student registered successfully!

Name : Ram
Roll No : 101
Father Name: Kumar
Age : 23
Gender : Male
Course : Java
Hobbies : Drawing
Address : 234 - 1d First Street, Anna Main Road, Namakkal

OK

Pop-up shown after clicking Save Details

Q7. Calculator Form — Event-Driven Programming

Scenario: Create a simple calculator GUI to perform addition, subtraction, multiplication, and division between two numbers.

PROGRAM STRUCTURE

- Use **JFrame**, **TextField**, and **Button**
- Add event handling for each operator button
- Display the result in a text field or label

CODE · Calculator.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {
    private JTextField display = new JTextField(14);
    private boolean justEvaluated = false;
    private String lastResult = "";

    Calculator() {
        super("Calculator");
        display.setEditable(false);
        display.setHorizontalAlignment(JTextField.LEFT);

        JPanel top = new JPanel();
        top.setBackground(Color.BLUE);
        top.add(display);

        String[] labels = {"+", "1", "2", "3",
                           "-", "4", "5", "6",
                           "*", "7", "8", "9",
                           "/", ".", "0", "C"};

        JPanel grid = new JPanel(new GridLayout(4, 4, 6, 6));
        grid.setBackground(Color.BLUE);
        for (String s : labels) {
            JButton b = new JButton(s);
            b.addActionListener(this);
            grid.add(b);
        }

        JPanel bottom = new JPanel();
        bottom.setBackground(Color.BLUE);
        JButton eq = new JButton("=");
        eq.addActionListener(this);
        bottom.add(eq);

        JPanel center = new JPanel(new BorderLayout());
        center.setBackground(Color.BLUE);
        center.setBorder(BorderFactory.createEmptyBorder(4, 10, 4, 10));
        center.add(grid, BorderLayout.CENTER);

        add(top, BorderLayout.NORTH);
        add(center, BorderLayout.CENTER);
        add(bottom, BorderLayout.SOUTH);

        setSize(300, 330);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    public void actionPerformed(ActionEvent e) {
        String cmd = e.getActionCommand();

        if (cmd.equals("C")) {
            display.setText("");
            justEvaluated = false;
        } else if (cmd.equals("=")) {
            evaluate();
        } else if (isOperator(cmd.charAt(0))) {

```

```

        if (justEvaluated) {                                // continue from last result
            display.setText(lastResult);
            justEvaluated = false;
        }
        String text = display.getText();
        if (text.isEmpty()) {
            if (cmd.equals("-")) display.setText("-"); // negative first number
        } else if (operatorIndex(text) == -1 && !text.equals("-")) {
            display.setText(text + cmd);
        }
    } else {                                                // digit or '.'
        if (justEvaluated) {
            display.setText("");
            justEvaluated = false;
        }
        display.setText(display.getText() + cmd);
    }
}

private boolean isOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

// index of the operator (ignoring a leading minus sign), or -1
private int operatorIndex(String text) {
    for (int i = 1; i < text.length(); i++) {
        if (isOperator(text.charAt(i))) return i;
    }
    return -1;
}

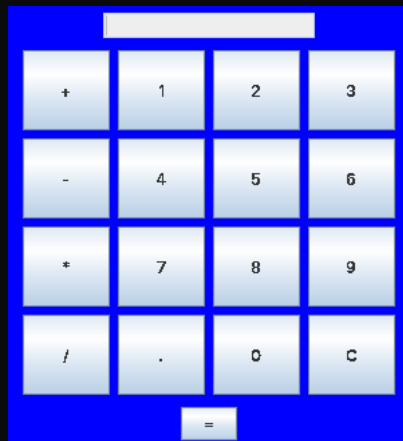
private void evaluate() {
    String text = display.getText();
    int i = operatorIndex(text);
    if (i == -1 || i == text.length() - 1) {
        return; // nothing to calculate yet
    }
    try {
        double a = Double.parseDouble(text.substring(0, i));
        char op = text.charAt(i);
        double b = Double.parseDouble(text.substring(i + 1));
        double result;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            default:
                if (b == 0) {
                    display.setText("Cannot divide by zero");
                    justEvaluated = true;
                    lastResult = "";
                    return;
                }
                result = a / b;
        }
        lastResult = String.valueOf(result);
        display.setText(text + "=" + lastResult);
        justEvaluated = true;
    } catch (NumberFormatException ex) {
        display.setText("Invalid input");
        justEvaluated = true;
        lastResult = "";
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new Calculator().setVisible(true));
}
}

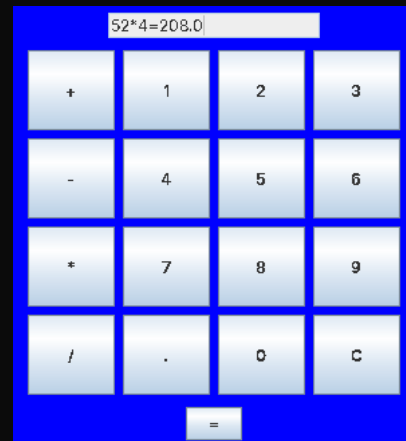
```

OUTPUT

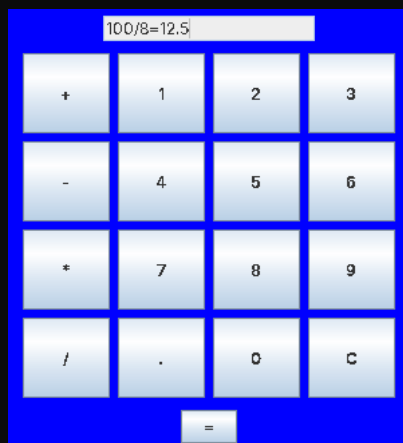
```
student@java-lab:~/module4$ javac Calculator.java
student@java-lab:~/module4$ java Calculator
# Calculator window opens (JFrame) – results below:
```



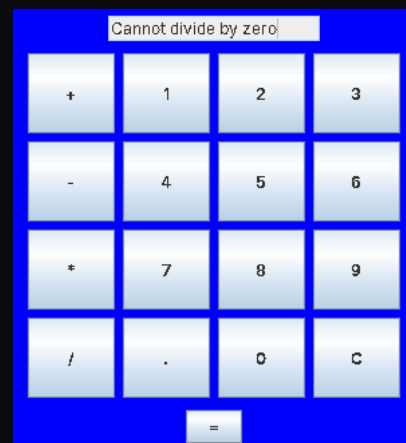
On start



52 * 4 = 208.0



100 / 8 = 12.5



7 / 0 → Cannot divide by zero